

TaRZI

Unified Android Super-App Architecture Specification

One application. Three major systems.
TaRZI Protection + TaRZI Assistant + Velune Music

Area	Design decision
Product identity	TaRZI is the complete application; the assistant is not a separate product.
Protection	Deterministic, offline-capable, defense-in-depth content filtering.
Assistant	Natural-language command system with strict capability boundaries.
Music	Existing Velune repository integrated as an internal media subsystem.
Network	Device enforcement by default; whole-network enforcement only through an authorized gateway/router.
AI	Optional reasoning/classification layer; never the final security authority.

Prepared as an implementation-grade architecture baseline for an autonomous coding agent.

Document Map

This document is intentionally implementation-oriented. It describes the target architecture, module boundaries, command and policy contracts, integration workflow for a GitHub-only Velune source tree, security model, Android limitations, testing strategy, and release gates.

Section	Focus
1	Executive Architecture
2	Existing-System Baseline
3	Velune Acquisition and Integration
4	Unified Project Structure
5	Core Runtime and Shared Services
6	TaRZI Assistant Architecture
7	Command Registry and Tool Contracts
8	Permission, Risk, and Confirmation Model
9	Protection Engine Architecture
10	Global Explicit-Content Policy
11	Classification and AI Escalation
12	VPN, DNS, and Network Architecture
13	Velune Media Architecture
14	TaRZI Memory and Context
15	Automation Engine
16	Android System Capabilities
17	External Integrations
18	Data Model and Persistence
19	Resilience and Failure Recovery
20	Tamper Resistance and Trust Model
21	Privacy and Secure Credential Handling
22	Testing and Bypass Matrix
23	Build, CI, and Release Engineering
24	Implementation Roadmap
25	Acceptance Criteria and Definition of Done

Current external reference. Velune is a public GitHub repository described as an Android YouTube Music client built with Kotlin and Jetpack Compose. Its repository currently exposes modules including app, betterlyrics, canvas, innertube, kizzy, kugou, and others; its README states MVVM + Clean Architecture + UDF, with Media3/ExoPlayer, Hilt, Room, Ktor/Retrofit, Coroutines/Flow, and Gradle KTS. It is licensed GPL-3.0. See the source notes at the end of this document.

1. Executive Architecture

TaRZI is defined as a single Android application that combines three major user-facing capabilities: a deterministic content-protection system, a natural-language assistant, and the integrated Velune music experience. The design intentionally prevents the assistant from becoming a privileged shortcut around the protection engine or Android security model.

The system should feel unified in the UI while remaining modular underneath. The security engine owns policy truth; the assistant translates user intent into typed commands; Velune owns playback state; shared services provide persistence, events, permissions, credentials, notifications, diagnostics, and auditing.

The implementation strategy is integration-first rather than rewrite-first. Existing blocker components that are already working should be preserved and hardened. Velune should be cloned from GitHub, audited, then integrated through explicit module and API boundaries rather than copied indiscriminately.

Core rule: TaRZI can plan an action, but only a deterministic executor can perform it. The AI cannot directly call arbitrary Android APIs, execute shell commands, bypass permissions, or override security policy.

Target system flow

User -> voice/text -> intent engine -> typed command -> schema validation -> permission gate -> risk/policy gate -> confirmation -> deterministic executor -> verification -> audit -> response.

```
TaRZI
├── Core
├── Protection
├── Assistant
├── Velune
└── Integrations
```

2. Existing-System Baseline

The provided baseline already contains a substantial ShieldX-style blocker architecture: a local VPN service, DNS filtering, SafeSearch enforcement, packet parsing, domain matching, classifiers, Device Owner controls, admin recovery, tamper classes, policy engine, audit logging, and a Compose UI. The implementation history also shows iterative compile/test cycles and dedicated architecture/security documentation.

The main architectural risk is not missing features; it is accidental duplication and policy conflicts when Velune and the assistant are introduced. The existing protection engine should therefore remain the authoritative policy layer, while TaRZI becomes an orchestration client of that engine.

The build environment described previously uses Gradle 8.9 and a locally available Android SDK path. The next build automation should verify the live environment rather than assuming these versions remain valid.

Existing component	Role in final TaRZI	Keep / adapt / replace
ProtectionVpnService	Device-level packet/DNS enforcement	Keep and adapt
DnsFilter / DomainMatcher	Fast deterministic domain decisions	Keep
SafeSearchEnforcer	Search restriction layer	Keep
Classifiers	Local content classification	Keep and expose policy API
DeviceControllers	Managed-device authority	Keep and isolate
AuditLogger	Security/accountability trail	Generalize to shared audit service
GeminiClassifier	Uncertain-content classifier	Keep as optional classification tool, not authority
Dashboard/UI	Existing management experience	Reframe as TaRZI protection screens where applicable

3. Velune Acquisition and Integration

Velune is currently available to the project as a GitHub repository rather than an installed local module. The coding workflow must therefore treat repository acquisition as a first-class build stage: check Git availability, clone the repository into an external integration workspace, verify the clone, inspect its build configuration, and only then plan the merge.

The current public repository structure includes the main app plus modules such as betterlyrics, canvas, innertube, kizzy, kugou, lastfm, and related components. The README states Kotlin + Jetpack Compose + Material 3, Media3/ExoPlayer, Hilt, Room (encrypted), Ktor + Retrofit, Coroutines + Flow, and Gradle KTS. The project also documents Android Studio Ladybug+, JDK 17, and Android SDK 34 as its local development baseline. These are facts to use as an initial compatibility target, not hard-coded assumptions. [Sources: Velune GitHub repository and README]

Velune is explicitly licensed GPL-3.0. The integrated project must preserve applicable notices and evaluate the consequences of distributing a combined derivative work. This is a release-engineering constraint, not a comment to postpone until the store submission step. [Source: Velune GitHub repository]

Audit target	Questions to answer before merge
Build	Which AGP/Kotlin/Gradle/SDK versions are declared?
Modules	Which modules are app-facing vs support libraries?
Media	Where is Media3/ExoPlayer/MediaSession initialized?
Storage	What encrypted Room database exists?
Networking	Which services use Ktor, Retrofit, or InnerTube?
Secrets	Which local.properties or IDs are required?
Licensing	Which code and dependencies carry GPL/other obligations?

```
# Acquisition stage
git --version
git clone https://github.com/nikhilvishwakarma00/Velune.git external/velune

# Initial inspection
cd external/velune
./gradlew tasks
./gradlew dependencies
./gradlew assembleDebug
```

4. Unified Project Structure

The final repository should have one Android application identity and shared infrastructure. Package names may remain descriptive; user-facing naming must consistently be TaRZI. The implementation should prefer a small number of stable interfaces over deep cross-module reach-through.

```
app/  
├── core/  
│   ├── database/  
│   ├── permissions/  
│   ├── settings/  
│   ├── events/  
│   ├── notifications/  
│   ├── credentials/  
│   └── audit/  
├── protection/  
│   ├── vpn/  
│   ├── dns/  
│   ├── policy/  
│   ├── classifier/  
│   ├── apps/  
│   └── network/  
├── assistant/  
│   ├── voice/  
│   ├── intent/  
│   ├── planning/  
│   ├── commands/  
│   ├── memory/  
│   └── automation/  
├── velune/  
│   ├── player/  
│   ├── session/  
│   ├── playlists/  
│   ├── lyrics/  
│   └── library/  
└── integrations/  
    ├── contacts/  
    ├── calendar/  
    ├── reminders/  
    ├── messaging/  
    └── shopping/
```

5. Core Runtime and Shared Services

The Core layer is the glue that prevents each subsystem from inventing its own storage, permissions, events, and lifecycle conventions. It should be deliberately boring: typed interfaces, clear ownership, deterministic state transitions, and no AI logic.

Use an application-scoped service registry or dependency-injection graph to expose stable controllers. Prefer repositories and use cases over direct database access from UI or assistant code.

Service	Primary responsibility	Key invariant
SettingsManager	User and system preferences	Validated values only
PermissionManager	Current permission state	No stale authorization assumptions
EventBus	Cross-subsystem events	Typed events with ownership
ServiceManager	Start/stop/reconcile long-lived services	Idempotent lifecycle
CredentialManager	Tokens and secrets	Encrypted, never logged
AuditService	Security and tool execution events	Append-only semantics where practical
NotificationManager	Channels and user-facing status	No duplicate channel creation
DiagnosticsService	Health and support snapshots	No secret leakage

```
interface TaRZIController {
    suspend fun execute(request: ToolRequest): ToolResult
    fun capabilities(): Set<Capability>
}

interface ProtectionController : TaRZIController {
    suspend fun setProfile(profile: ProtectionProfile): Result<Unit>
}

interface VeluneController : TaRZIController {
    suspend fun play(command: PlaybackCommand): Result<Unit>
}
```

6. TaRZI Assistant Architecture

The assistant is a controlled orchestration layer. It is allowed to interpret natural language, resolve ambiguity, build a plan, and select a registered tool. It is not allowed to invent capabilities outside the registry.

A short-lived Context Engine provides current application/media/network state. Persistent Memory is separate and user-controlled. This separation prevents a casual sentence from silently becoming a permanent memory and prevents stale context from being treated as fact.

Model boundary: A cloud or local language model is a planner/parser. The application remains the authority. Tool execution must happen in deterministic application code.

Assistant component	Responsibility
IntentEngine	Map speech/text to structured intents
Planner	Break multi-step requests into tool calls
CommandRegistry	Define allowed capabilities
PermissionGate	Check Android/service permissions
RiskEngine	Classify action sensitivity
ConfirmationManager	Require explicit approval when needed
Executor	Call deterministic subsystem APIs
Verifier	Confirm real post-action state
ResponseEngine	Describe actual result, not intended result

```
Input
-> Speech/Text Normalizer
-> Intent Engine
-> Planner
-> Tool Registry
-> Schema Validator
-> Permission Gate
-> Risk/Policy Gate
-> Confirmation Manager
-> Executor
-> Verification
-> Audit
-> Response
```


7. Command Registry and Tool Contracts

Every TaRZI capability should be represented by a versioned tool contract. A tool must declare its name, parameters, permissions, risk, confirmation rule, timeout, and deterministic handler. This makes the command surface auditable and testable.

Tool family	Examples
Device	GET_BATTERY, GET_STORAGE, LOCK_DEVICE, OPEN_SETTINGS
Protection	ENABLE_PROTECTION, SET_PROFILE, GET_STATUS, BLOCK_APP
Velune	PLAY, PAUSE, NEXT, PREVIOUS, PLAY_PLAYLIST, QUEUE
People	FIND_CONTACT, CALL_CONTACT, SEND_MESSAGE
Memory	MEMORY_WRITE, MEMORY_READ, MEMORY_DELETE
Planning	CREATE_REMINDER, CREATE_EVENT, CREATE_AUTOMATION
Network	GET_WIFI, SET_NETWORK_PROFILE, GET_NETWORK_STATUS
Commerce	SEARCH_PRODUCTS, COMPARE_PRODUCTS, PURCHASE

```
ToolDefinition {
  name: String
  version: Int
  inputSchema: JsonSchema
  requiredPermissions: Set<Permission>
  riskLevel: RiskLevel
  confirmation: ConfirmationPolicy
  timeoutMs: Long
  handler: ToolHandler
  auditCategory: AuditCategory
}
```

8. Permission, Risk, and Confirmation Model

The security model uses both Android permissions and application-level risk. Having an Android permission is not automatically sufficient authorization for a high-impact assistant command. For example, purchase and destructive operations should be protected by an application policy even if an integration token is valid.

Confirmation contract

The confirmation prompt must include the resolved target and the effective consequence. For money or irreversible actions, confirmation must occur immediately before the final operation, not several steps earlier.

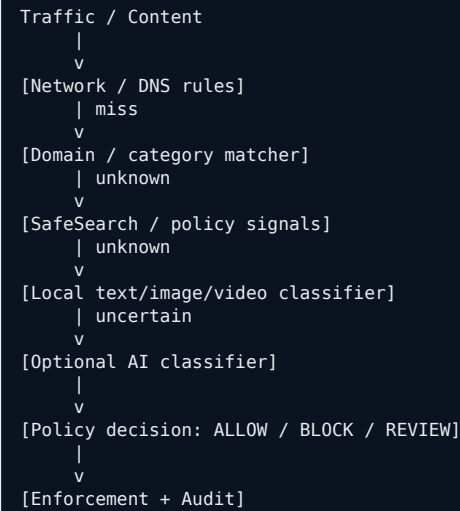
Risk	Typical actions	Default confirmation
L0	Read-only device/media state	None
L1	Open app, play/pause, non-destructive UI actions	None
L2	Call, send message, create reminder	Configurable / context dependent
L3	Change protection profile, block apps, managed-device settings	Required
L4	Purchase, delete data/account, irreversible action	Always required

```
if (!permissionGate.allows(request)) return DENIED
if (!policyEngine.allows(request)) return DENIED
if (confirmation.required(request) && !confirmation.approved(request))
    return REQUIRES_CONFIRMATION
return executor.execute(request)
```

9. Protection Engine Architecture

The protection engine remains the security authority. Its core pipeline is deterministic and layered so that failure or ambiguity in one layer does not disable the entire system. The design should prefer cheap high-confidence decisions first and expensive classification last.

Layer	Purpose	Expected behavior on failure
DNS/domain	Fast, low-cost filtering	Continue to next rule / fail closed where policy requires
SafeSearch	Search result restriction	Remain enabled if configured
Local classification	Private, deterministic/ML classification	Use fallback rule set
AI classifier	Resolve ambiguous cases	Trip circuit breaker; do not weaken baseline policy
Enforcement	Apply final decision	Verify actual blocking state



10. Global Explicit-Content Policy

The command "TaRZI, block porn" should be treated as a policy profile change, not as a single domain-list toggle. It activates the strongest supported explicit-content policy across the application's enforcement surfaces. The exact coverage is platform-dependent and must be reported honestly.

Fail-closed principle: When a mandatory protection mechanism is unexpectedly unavailable, the system should prefer disabling the affected capability or showing a protective error over silently reporting protection that is not active.

- Browsers: DNS/domain filtering, search controls, and supported URL/content classification.
- Social/video apps: network and supported app policy coverage; direct private app content is not universally inspectable on Android.
- Images/text: local classification where TaRZI has legitimate visibility; AI only when explicitly enabled and privacy-compatible.
- Velune: route discovery/playback surfaces through the same policy contract; do not create a hidden unrestricted path.
- Network-level: device VPN for the phone; whole-network control only through an authorized gateway/router configuration.
- Managed mode: Device Owner controls can add stronger app-policy and device-admin enforcement where legitimately provisioned.

```
ProtectionProfile.MAXIMUM =  
  dnsFiltering = ON  
  safeSearch = ENFORCED  
  explicitCategoryBlocks = ON  
  localClassification = ON  
  aiEscalation = OPTIONAL  
  velunePolicy = ENFORCED  
  tamperMonitoring = ON  
  networkProfile = AUTHORIZED_PROFILE_ONLY
```

11. Classification and AI Escalation

AI classification is useful for uncertain content but should not be the single point of failure. The classifier pipeline should expose confidence and provenance. The policy engine decides what to do with that result.

AI controls

Use strict JSON schema validation, request size limits, timeouts, rate limits, privacy settings, circuit breakers, and cache policies. Never treat malformed or unavailable AI output as permission to bypass a block.

Signal	Example	Owner
Deterministic domain	Known explicit domain	DomainMatcher
Text model	Caption or OCR-like text score	Local classifier
Image model	Explicitness probability	Local image classifier
Video model	Frame/metadata score	Local video classifier
Cloud AI	Ambiguous case with approved data path	AI tool/controller

```
AIDecision {
  label: EXPLICIT | NON_EXPLICIT | UNCERTAIN
  confidence: Float
  model: String
  policyVersion: String
  expiresAt: Instant?
}
```

12. VPN, DNS, and Network Architecture

TaRZI can strongly protect the device that runs it using an Android VPN service that captures relevant traffic and applies local DNS/domain policies. This is distinct from controlling every other device on the same Wi-Fi network. A phone app cannot automatically acquire authority to inspect arbitrary peers simply by learning the SSID.

Scenario	TaRZI capability	Authorization requirement
Phone on any Wi-Fi	Device-level filtering	Normal VPN permission
Phone on home Wi-Fi	Device filtering + optional gateway policy	User controls gateway
Third-party public Wi-Fi	Device filtering only	No network-wide authority
Managed device fleet	Stronger device controls	Device Owner / Enterprise provisioning

Device path

Phone apps -> TaRZI VPN -> DNS / policy engine -> network

Authorized network path

Client devices -> user-controlled router/gateway -> TaRZI-compatible DNS/policy -> Internet

Network detection

Wi-Fi connected -> identify network -> resolve stored profile -> activate device policy

Wi-Fi changed -> revoke previous network context -> resolve new profile

13. Velune Media Architecture

Velune should be integrated through its existing media architecture rather than replacing it with a new player. The current repository documents Media3/ExoPlayer, MediaSession, background playback, playlists, encrypted offline caching, lyrics, audio processing, and related modules. Its contributor guidance describes a clean separation between UI, domain, data, and service layers.

[\[cite\]](#)[turn383101search0](#)[turn383101search2](#)

Velune remains an internal subsystem of TaRZI. It is not a separate installed application in the final user experience.

Integration contract

Expose a small internal controller interface so TaRZI does not depend on Velune UI internals. The assistant should be able to ask for playback state, enqueue media, play named playlists, pause/resume, and adjust supported playback options.

```
TaRZI Assistant
|
v
VeluneController
|
+--> MediaController / MediaSession
|
+--> Player / Queue
|
+--> Playlist / Library
|
+--> Lyrics / Discovery
|
+--> Audio processing
```

14. TaRZI Memory and Context

Memory should be explicit, auditable, and user-controlled. Use short-lived context for conversational references and a separate persistent memory store for user-approved facts. Avoid storing entire conversation logs as permanent memory by default.

Memory type	Lifetime	Example
Turn context	Seconds/minutes	"Play that again" -> current track
Session context	App session	Current Wi-Fi, current screen
Preference memory	Persistent until removed	Favorite pizza, favorite playlist
Routine memory	Persistent	Morning briefing routine
Sensitive memory	Persistent but protected	Only when explicitly requested and securely stored

```
MEMORY_WRITE
  key = favorite_pizza
  value = paneer pizza
  source = USER_EXPLICIT
  consent = true

MEMORY_DELETE
  key = favorite_pizza
```


15. Automation Engine

Automation turns TaRZI from a command-only assistant into a routine engine. Each automation has a trigger, optional conditions, a sequence of actions, and a policy snapshot. Actions still pass through the same capability and permission model used by interactive commands.

- Wi-Fi connect/disconnect
- Bluetooth/headset events
- Time schedules
- Charging state
- Application lifecycle events where Android permits observation
- Media events
- Device boot and recovery states

```
Automation {  
  id  
  trigger  
  conditions[]  
  actions[]  
  enabled  
  createdByUser  
  policyVersion  
}
```

Example:

TRIGGER: WiFiConnected(HOME)

ACTIONS:

1. SET_PROTECTION_PROFILE(HOME)
2. PLAY_PLAYLIST(EVENING)

16. Android System Capabilities

TaRZI should use Android-supported mechanisms rather than trying to simulate root access. Capability strength depends on the mode in which the device is operating.

Capability	Typical mechanism	Notes
Call contact	Contacts + ACTION_CALL / supported telephony APIs	Requires permission and device support
Open app/settings	Explicit intents	Do not assume every intent is exported
Media control	Media3 MediaSession / MediaController	Ideal for Velune integration
Device lock	DevicePolicyManager where authorized	Requires appropriate management mode
App management	DevicePolicyManager / package visibility rules	Stronger in managed mode
UI observation	AccessibilityService	Explicit user enablement and policy constraints
Network filtering	VpnService / DNS logic	Device scope, not automatic peer control
Notifications	Notification APIs / listeners where permitted	Avoid storing sensitive content unnecessarily

17. External Integrations

External services should be connected through scoped adapters. Search and product discovery can be read-only until the user chooses an action. Purchasing, sending messages, and other consequential operations must pass confirmation and authorization checks.

External credentials must never appear in prompts, source files, audit logs, analytics payloads, or model memory. Use secure storage and provider-scoped tokens.

Integration	Read capability	Write/action capability	Protection
Contacts	Resolve names	Call/message	Permission + confirmation
Calendar	Read events	Create/update	Permission
Reminders	Read reminders	Create/delete	Permission
Maps	Search places	Open navigation	Permission as needed
Shopping	Search/compare	Purchase	Always-confirm transaction
Email	Read with explicit provider permission	Send	Confirmation + recipient check

18. Data Model and Persistence

Use one persistence architecture for shared TaRZI state where practical. Existing Velune and blocker stores should be evaluated for consolidation versus encapsulation. Avoid merging schemas simply because both use Room; merge only where shared lifecycle and ownership are real.

Data	Owner	Retention
Protection configuration	Core/Protection	Persistent
Audit events	Core/Audit	Policy-defined
Memory items	Assistant/Memory	Until user deletes / retention policy
Playback state	Velune	Persistent session state where needed
Credentials	CredentialManager	Provider-controlled expiration

```
Core entities
- UserPreference
- ProtectionProfile
- NetworkProfile
- AppPolicy
- MemoryItem
- Automation
- AuditEvent
- PermissionState
- ToolDefinition

Velune-owned entities remain isolated where their schema is intrinsic to playback/library operation.
```

19. Resilience and Failure Recovery

Every long-lived service must have an explicit health model. Recovery should be idempotent: starting an already-started service should not create duplicate tunnels, sessions, notifications, or schedulers.

Failure	Detection	Recovery
VPN crash	Service/process monitor	Restart; reconcile state; verify tunnel
DNS provider failure	Timeout/error rate	Use local deterministic cache/rules
AI timeout	Request timeout/circuit breaker	Fall back to local policy
Velune playback error	MediaSession/player state	Reset/reprepare player; report actual state
Database exception	Transaction failure	Retry safe operations; enter degraded mode for unsafe ones
Wi-Fi change	Connectivity callback	Recompute network profile and policy
Reboot	Boot receiver/service startup	Restore state; do not claim enabled until verified

HealthState:

UNKNOWN
STARTING
HEALTHY
DEGRADED
FAILED
RECOVERING

Safety rule: UI state is derived from verified service state, not from a button click event.

20. Tamper Resistance and Trust Model

Protection should resist accidental shutdown and obvious tampering without pretending that a normal Android application is unbreakable. Stronger enforcement should be tied to managed-device provisioning. The trust model should distinguish user-authorized configuration, service health, and managed-device authority.

No anti-tamper design should claim absolute persistence against a factory reset, OS reinstallation, unlocked bootloader, or a device owner with legitimate authority to wipe/reprovision the device.

- Maintain integrity checks for critical configuration files and policy snapshots.
- Watch critical service state and reconcile after restart.
- Keep an admin recovery path that is itself protected and auditable.
- In managed mode, use Device Owner capabilities where legitimate.
- Never grant the AI a bypass role or hidden recovery credential.
- Record protection profile changes and security-sensitive command results.

21. Privacy and Secure Credential Handling

TaRZI should follow a local-first approach: do as much protection, command routing, and media control on-device as possible. Cloud AI should receive only the minimum data required for the specific operation, and only when the chosen AI provider and privacy settings permit it.

- Do not upload raw media by default for classification.
- Prefer domain/category decisions before content-level AI.
- Redact or minimize logs that contain user content.
- Encrypt persistent sensitive memory and credentials.
- Use token-based provider authorization instead of storing passwords.
- Provide user-visible controls for memory, AI use, and connected services.
- Keep diagnostic exports free of secrets by default.

```
Credential boundary
Provider OAuth -> secure storage -> integration adapter
|
+--> never exposed to model prompt
+--> never persisted in audit logs
+--> scoped to provider capability
```

22. Testing and Bypass Matrix

The test strategy is intentionally broader than ordinary unit testing. The app is simultaneously a media player, system assistant, and security product, so tests must prove integration behavior and degradation behavior.

Test family	Representative tests	Pass criterion
Command parser	"play my workout playlist"	Correct typed tool request
Security	"block porn"	Maximum explicit policy activated
Permission	missing CALL permission	Command denied, truthful response
Confirmation	purchase action	No purchase before explicit confirmation
VPN	service restart	Policy restored and verified
Network	Wi-Fi switch	Correct profile selected
Velune	pause/next/background	Actual MediaSession state matches response
AI	timeout/malformed JSON	Fallback; no policy bypass
Reboot	boot into protected profile	State reconciles before reporting active
Tamper	attempt to change protected policy	Blocked or detected according to mode

```
Required validation loop
1. Unit tests
2. Instrumentation tests
3. Integration tests
4. Fault injection
5. Bypass tests
6. Manual acceptance flows
7. Release build verification
```


23. Build, CI, and Release Engineering

The build pipeline must automatically establish what is present rather than assuming a particular workstation state. Because Velune is GitHub-only at project start, repository clone and build verification are explicit pipeline stages. The current Velune contributor guidance also documents the Gradle wrapper, assembleDebug/assembleRelease, lint, formatting, and unit-test expectations. [Source: Velune CONTRIBUTING.md]

Gate	Must be true before release
Compilation	No build errors; all modules resolve
Tests	Required unit/integration suites pass
Lint	No release-blocking lint defects
Security	No known critical bypass regression
Licensing	Required notices/source obligations documented
APK/AAB	Artifact exists and installs in validation environment
Runtime	Protection, assistant, and Velune smoke tests pass

```
# Environment discovery
java -version
git --version
./gradlew --version

# Acquire external source
git clone https://github.com/nikhilvishwakarma00/Velune.git external/velune

# Validate before merge
cd external/velune
./gradlew testDebugUnitTest
./gradlew assembleDebug

# Unified project gates
cd ../../
./gradlew clean
./gradlew test
./gradlew testDebugUnitTest
./gradlew lint
./gradlew assembleDebug
./gradlew assembleRelease
```

24. Implementation Roadmap

The implementation should proceed in controlled phases. Each phase ends with a build/test checkpoint. Do not perform a large merge and then debug dozens of interacting failures simultaneously.

Phase	Work	Exit condition
0	Workspace + environment audit	Tools and current code mapped
1	Velune clone + license/dependency audit	Standalone Velune builds or blockers documented
2	Core consolidation	Shared DI/events/settings/audit stable
3	Velune integration	Playback and background media pass
4	Protection integration	VPN/DNS/policy regression suite passes
5	TaRZI command framework	Typed tools + policy gate operational
6	Memory + automation	Persistence and trigger tests pass
7	System/external integrations	Permissions and confirmations validated
8	Hardening	Fault injection and bypass suite passes
9	Release	APK/AAB, docs, notices, and final audit complete

Each phase:
AUDIT -> IMPLEMENT -> BUILD -> TEST -> FIX -> REBUILD -> VERIFY
Never skip the verification step.

25. Acceptance Criteria and Definition of Done

TaRZI is complete only when the integrated application demonstrates the promised behavior on a real Android validation device or emulator, the build is reproducible, and the security model is supported by tests. Documentation alone is not evidence of completion.

Definition of done: no feature is considered complete because files were generated. It is complete only after compilation, tests, runtime verification, failure tests, and documentation of any known limitation.

- The final application is branded and presented as TaRZI.
- The existing blocker remains functional and is the security authority.
- Velune is integrated into the same application rather than installed separately.
- TaRZI can parse and execute representative device, protection, and Velune commands.
- Memory and automation obey the same permission model.
- High-risk actions require confirmation.
- AI cannot execute arbitrary code or bypass security gates.
- Device-level filtering works independently of cloud AI availability.
- Network-wide filtering is implemented only through authorized network infrastructure.
- Velune playback remains stable across background/notification controls.
- Reboot, network changes, and service restarts are tested.
- The actual build result and APK/AAB path are verified.
- Licensing and source-attribution obligations are documented before distribution.

Appendix A - Command Catalog

Category	Examples
Protection	block explicit content; enable maximum protection; show blocked events; block app; allow app; show policy status
Device	battery; storage; Wi-Fi; Bluetooth; open settings; open app; lock device where authorized
Velune	play; pause; resume; skip; previous; search track; play playlist; queue; shuffle; repeat; favorite
Memory	remember; what do you remember; forget; update preference
People	call Mom; message John; find contact
Planning	remind me; create event; list calendar; create automation
Network	what Wi-Fi; network status; apply authorized network profile
Commerce	search products; compare options; purchase with confirmation

Example multi-step command

```
"TaRZI, connect to my home setup, enable maximum protection,
play my evening playlist, and remind me to call Mom at 8 PM."

Plan:
1. Resolve current network and home profile
2. Apply protection profile (confirmation if required by user policy)
3. Start Velune playlist
4. Create reminder
5. Verify each step
6. Report partial failure accurately if any step fails
```

Appendix B - Integration Guardrails

- Do not blindly merge manifests; review every service, receiver, provider, permission, and intent filter.
- Do not create duplicate MediaSession or playback services during Velune integration.
- Do not expose the security engine as a generic model tool.
- Do not put credentials in model context.
- Do not trust a model-generated tool result as evidence that an operation succeeded.
- Do not silently weaken filtering because an external AI service is unavailable.
- Do not promise whole-network filtering from a phone VPN alone.
- Do not remove GPL-3.0 notices or third-party attribution.
- Do not ship a release without verifying the actual output artifact.
- Do not report a feature as working until a test proves it.

Appendix C - Source Notes

1. Velune GitHub repository: <https://github.com/nikhilvishwakarma00/Velune> - repository overview, architecture, modules, tech stack, requirements, and GPL-3.0 statement. [Sources: Velune GitHub repository and README]
2. Velune contributing guidance: <https://github.com/nikhilvishwakarma00/Velune/blob/main/CONTRIBUTING.md> - architecture layers, environment initialization, Gradle build commands, and quality gates. [Source: Velune CONTRIBUTING.md]
3. Velune release page: <https://github.com/nikhilvishwakarma00/Velune/releases> - release history and latest published release information at the time of research. [Source: Velune GitHub releases]

Appendix D - Explicit Platform Boundary

The architecture is intentionally designed around supported Android capabilities. A normal application does not receive unrestricted visibility into every third-party app or every device on a Wi-Fi network. Device-level VPN/DNS filtering is distinct from router/gateway-level network control. Stronger fleet controls require authorized management modes such as Android Enterprise/Device Owner where applicable. The implementation should always communicate these boundaries truthfully.